

Multi-Agent Handoffs with LangGraph

A multi-agent system consists of multiple specialized agents that collaborate to solve a problem.

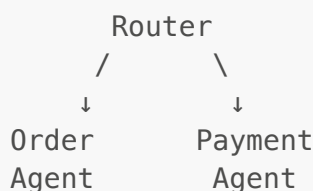
Consider an order-processing system with:

- **Order Agent** — handles general order requests.
- **Payment Agent** — handles payment-related issues.

The interesting part is not simply having two agents. It is allowing one agent to **decide when another agent should take over**.

1. Traditional Conditional Routing

A simple LangGraph workflow can use a router:



The application contains the routing logic:

```
if request_is_about_payment:
    return "payment"
```

This is centralized routing.

The agents themselves do not decide who should handle the request.

2. Agent-Based Handoff

In a multi-agent design, the Order Agent can analyze the request and decide whether another agent should handle it:



```
|
+----- order issue -----> Order Agent
```

The Order Agent is therefore participating in the routing decision.

3. Agent State

A simple shared state can contain:

```
class OrderState(TypedDict):
    order_id: str
    message: str
    response: str
    next_agent: str
```

The state is passed between agents.

For example:

```
{
    order_id: "ORD1001",
    message: "I was charged twice",
    ...
}
```

The Payment Agent can use the same `order_id` when it receives the handoff.

4. Let the LLM Decide

The Order Agent can ask the LLM to classify the request:

```
prompt = f"""
You are an Order Agent.

Customer request:
{state["message"]}

If this is payment-related, respond:
HANDOFF:PAYMENT

Otherwise respond:
HANDLE:ORDER
"""
```

The LLM might return:

```
HANDOFF:PAYMENT
```

The Order Agent converts this into a routing decision:

```
if decision == "HANDOFF:PAYMENT":  
    return {"next_agent": "payment"}
```

The important point is that the **semantic decision** came from the agent.

5. LangGraph Executes the Handoff

LangGraph can then use the agent's decision:

```
builder.add_conditional_edges(  
    "order_agent",  
    route_from_order,  
    {  
        "order": END,  
        "payment": "payment_agent"  
    }  
)
```

The resulting execution is:

```
START  
|  
v  
Order Agent  
|  
| "HANDOFF:PAYMENT"  
v  
Payment Agent  
|  
v  
END
```

LangGraph provides the execution mechanism, while the agent provides the decision.

6. Why This Is a Multi-Agent Pattern

The distinction is important.

In a conventional workflow:

```

Application Router
  |
  +-----> Agent A
  |
  +-----> Agent B

```

The router knows about the agents and decides where to send the request.

In a handoff-based system:

```

Agent A
  |
  | "This requires Agent B"
  v
Agent B

```

Agent A determines that another specialist is required.

This allows more dynamic collaboration.

7. Multiple Handoffs

The pattern can be extended:

```

      Order Agent
     /      \
    ↓        ↓
Payment Agent  Support Agent
    |
    ↓
  Fraud Agent

```

For example:

```

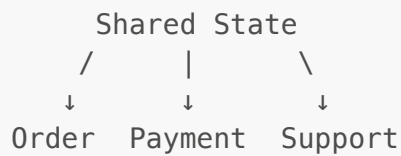
Order Agent
  ↓
Payment Agent
  ↓
  Fraud Agent

```

Each agent can specialize in a particular responsibility.

8. Shared vs. Isolated Context

Agents can share the same LangGraph state:



Alternatively, an application can provide each agent with only the information it requires.

For example, the Payment Agent might receive:

```
{
  "order_id": "ORD1001",
  "payment_issue": "charged twice"
}
```

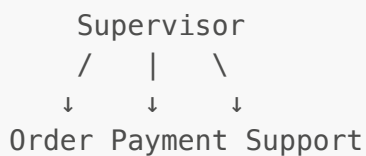
rather than the entire conversation.

This creates stronger separation between agents.

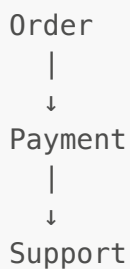
9. Centralized vs. Peer-to-Peer

These are two different topologies.

Centralized orchestration:



Peer-to-peer handoff:



The first uses a central decision-maker.

The second allows agents to transfer responsibility to one another.

10. Key Takeaway

The important concept is not simply "multiple agents."

It is **delegation between specialized agents**.

```
graph TD; A[User Request] --> B[Order Agent]; B --> C[Reason about request]; C --> D[Need another specialist?]; D --> E[Yes]; E --> F[Handoff]; F --> G[Payment Agent];
```

User Request
↓
Order Agent
↓
Reason about request
↓
Need another specialist?
↓
Yes
↓
Handoff
↓
Payment Agent

LangGraph provides the graph structure, state management, and execution flow. The LLM-powered agents provide the reasoning that determines when a handoff should occur.

This combination allows a workflow to move from **fixed routing** toward **dynamic multi-agent collaboration**.